

Hyperledger Sawtooth Overview

Sawtooth is an enterprise-grade distributed ledger for permissioned and permissionless networks.



Core Features of Sawtooth

Modular Design

Separates application logic from core blockchain system for flexibility.

Parallel Execution

Advanced scheduler enables parallel transaction processing for high throughput.

Global State Agreement

All nodes share a cryptographically verifiable, identical blockchain state.

Dynamic Consensus Mechanisms



Pluggable Consensus

Consensus algorithms can be changed while the blockchain runs.



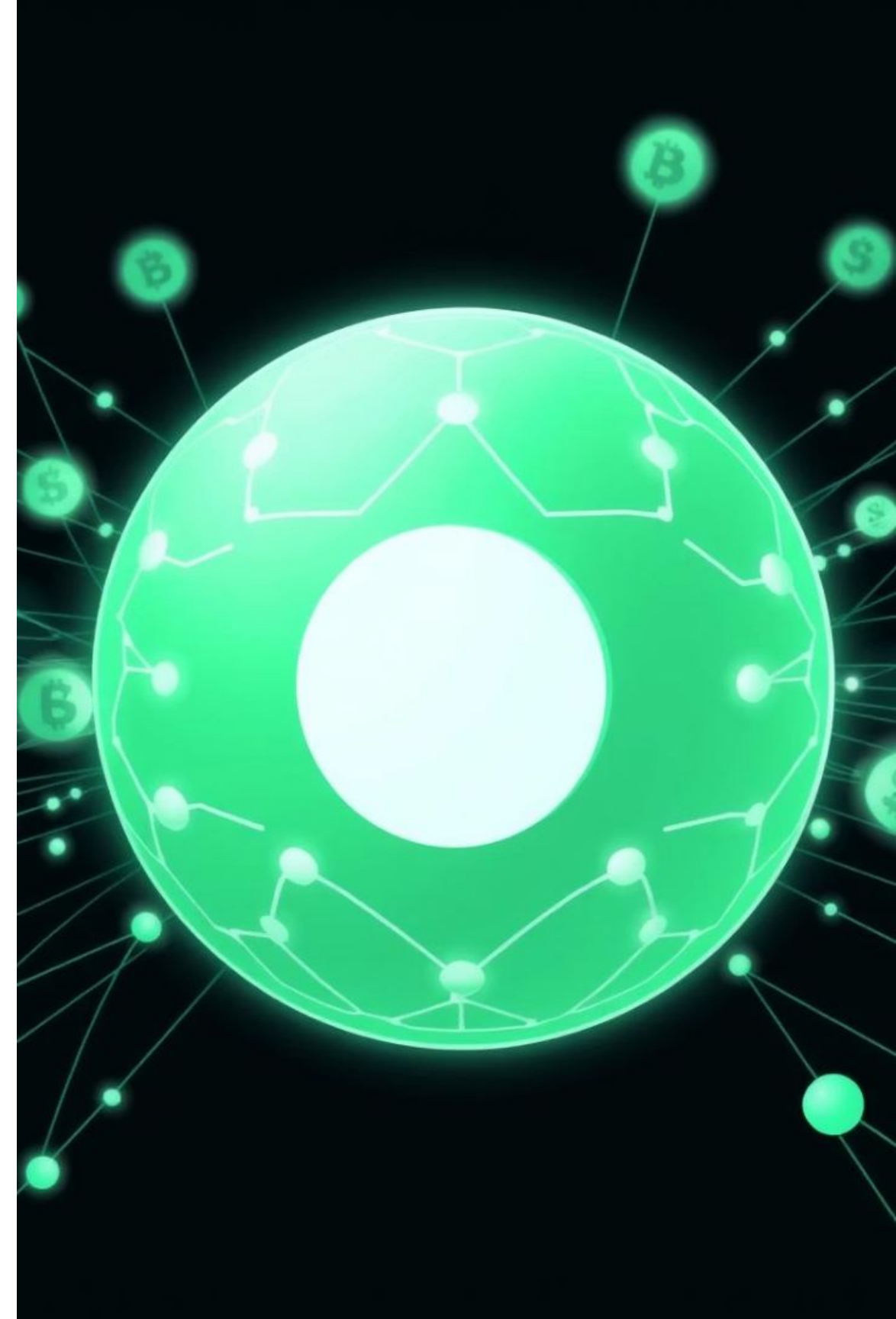
PoET Algorithm

Leader election based on random wait time using trusted execution environments.



Other Algorithms

Supports PBFT, Raft, and quorum voting for different network needs.



Proof of Elapsed Time (PoET)

Energy Efficient

Uses trusted computing instead of heavy resource consumption like PoW.

Intel SGX

Employs Intel's SGX for secure, random leader election (simulated in Sawtooth).

Limitations

Stale chip problem risks hardware wastage and potential security issues.

Transaction Lifecycle in Sawtooth

1

Client Submission

Client sends transaction batch to validator via REST API.

2

Propagation

Transaction spreads across validator network.

3

Leader Election

A validator creates and publishes a candidate block.

4

Validation & Execution

Validators and processors validate and execute transactions.

5

State Update

Validated block is stored and blockchain state is updated.

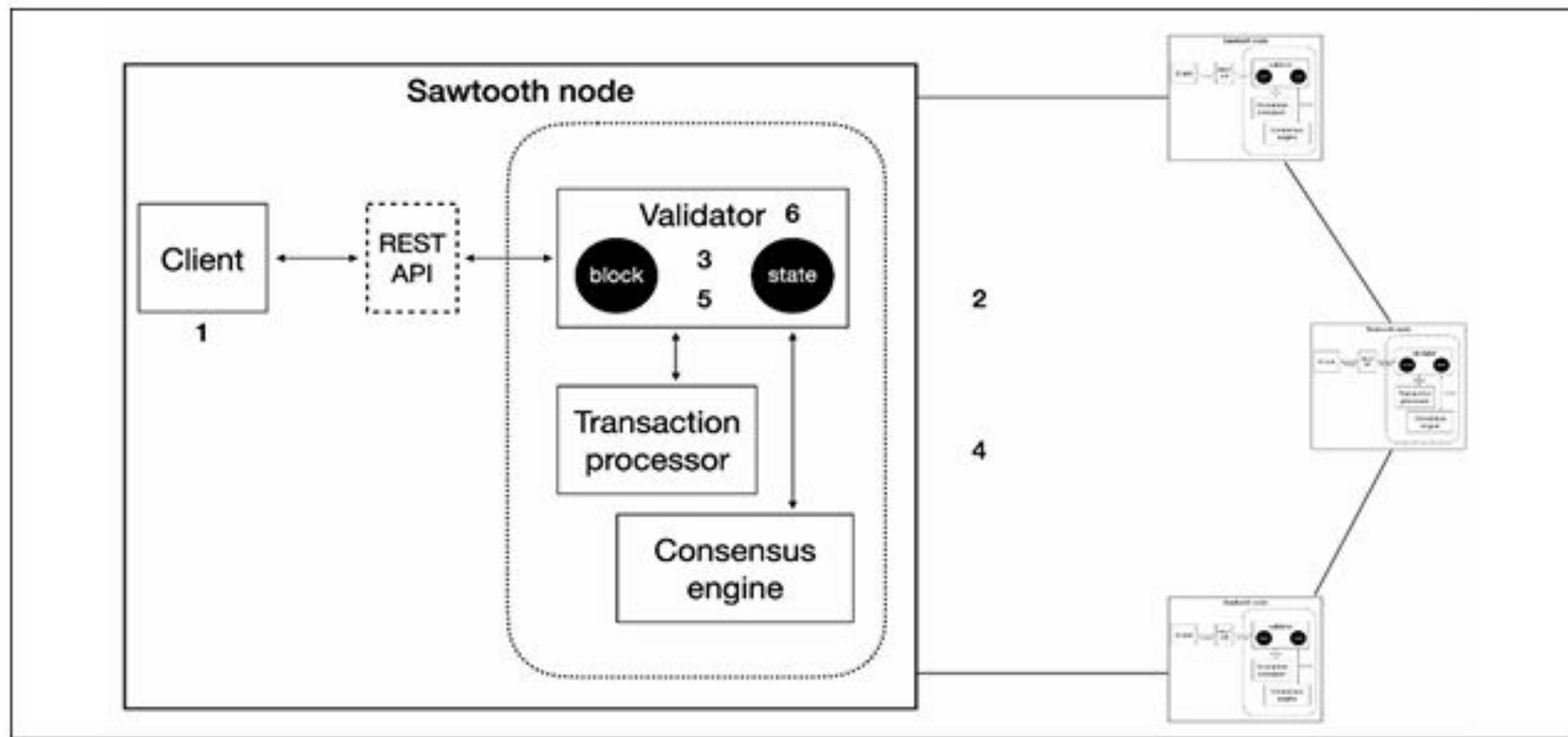


Figure 17.9: Sawtooth transaction lifecycle

Sawtooth Node Components

Validator

Validates transactions, creates blocks, and maintains consensus.

REST API

Interface between clients and validators over TCP port 8008.

Transaction Processors

Validate transactions and update blockchain state per rules.



Transaction Families Concept

Domain-Specific Logic

Transaction families define business rules for specific use cases.

Security Advantage

Limits attack surface by restricting operations to domain logic.

Customizable

Users build transaction families tailored to their business needs.

domains:

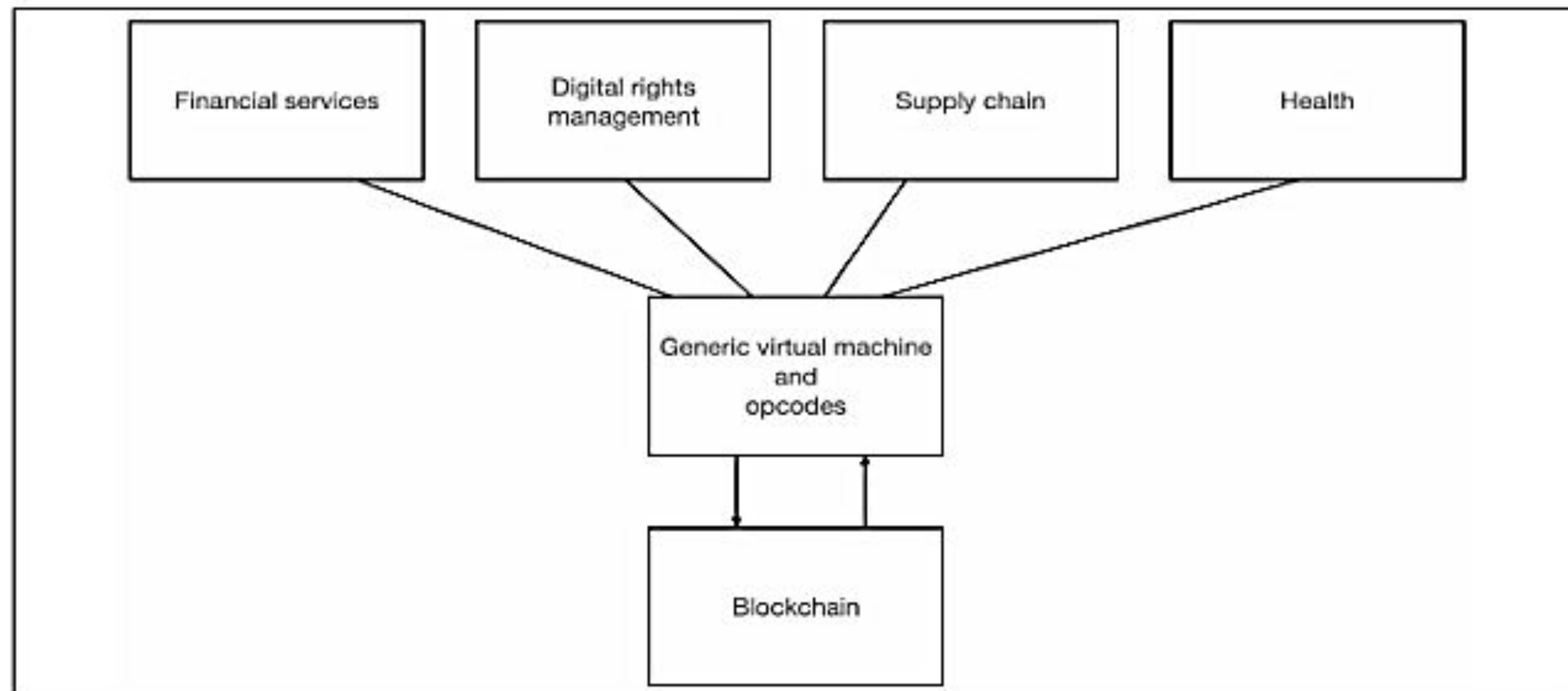


Figure 17.11: The traditional smart contract paradigm

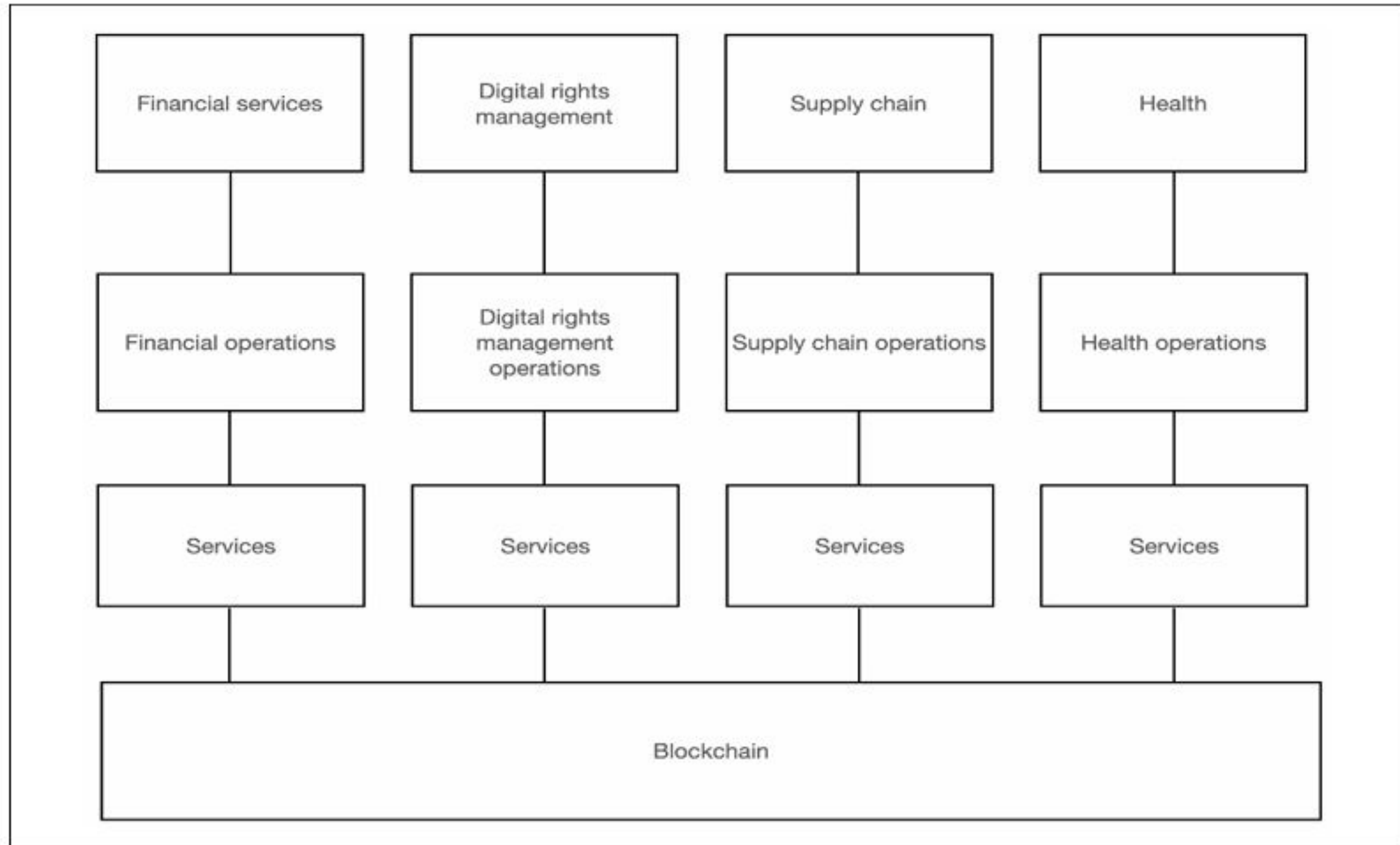
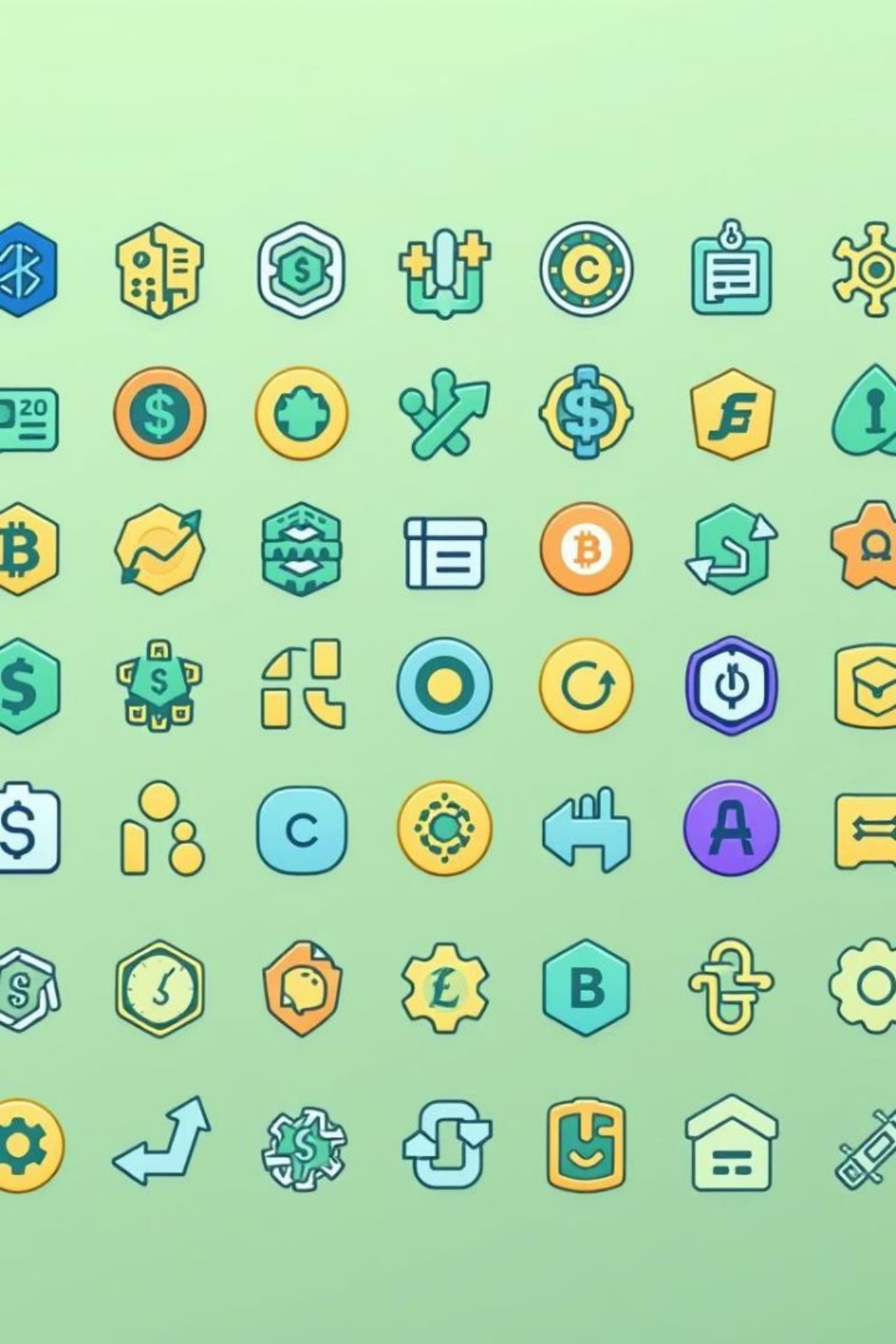


Figure 17.12: The Sawtooth (transaction families) smart contract paradigm

Examples of Transaction Families

BlockInfo	Stores historic block data	Python
Identity	Role and policy permissions	Python
Smallbank	Performance workload	Go
XO	Simple tic-tac-toe game	Go, JS, Python



Transaction Family Elements



Transaction Processor

Handles business logic, validation, and state updates.



Data Model

Defines data storage, valid operations, and payload structure.



Client

Creates, signs, batches, and submits transactions via REST API.



```

11  raly,
46  Sexal;
48  Ccontection, intledvalasd(Syalegy,
66  tniet dalafr,
87  inteartortant;
60  Inalegse.
28  "contwmes. "Wn259Egh.com.
87  "noctiation, furogay-udiny/rohatter",
28  constilation, "ogesatar",
48  covlectpar"lnfoi/t peroop",
75  cantal "ntraulorate (/ "inetfoclal gryort,
29  pastertdualeyn" Vips & wager feræeo"3"
36  "nett trle-mourkfe_page"
23  )

```



REST API and Practical Example

REST API enables communication between clients and transaction processors.

It can run on validator nodes or separately for flexible deployment.

Setting up a Sawtooth network

In this section, we will see how a Sawtooth network can be created. First we start with creating a directory and then we will download specific configuration files, which will help with the configuration of the network.

1. Create a directory named sawtooth:

```
$ mkdir sawtooth
```

2. Change the directory to sawtooth:

```
$ cd sawtooth
```

3. Download the PoET YAML file:

```
$ wget https://sawtooth.hyperledger.org/docs/core/nightly/1-2/app_developers_guide/sawtooth-default-poet.yaml
```

This command will show an output like the following, indicating that the sawtooth-default-poet.yaml file has been downloaded successfully. Note that only the final part of the output is shown for brevity:

```
...  
2020-06-29 21:04:22 (51.9 KB/s) - 'sawtooth-default-poet.yaml' saved  
[16543/16543]
```

4. Start the network:

```
$ docker-compose -f sawtooth-default-poet.yaml up
```

This will show a long output and will take several minutes to complete. The output will be similar to the following, which shows the progress of the process:

```
Creating network "sawtooth_default" with the default driver
Creating volume "sawtooth_poet-shared" with default driver
Pulling intkey-tp-0 (hyperledger/sawtooth-intkey-tp-python:chime)...
chime: Pulling from hyperledger/sawtooth-intkey-tp-python
```

5. Once the network is started up successfully, we can verify that all the components are up by simply checking it in the Docker dashboard, as shown in the following screenshot:

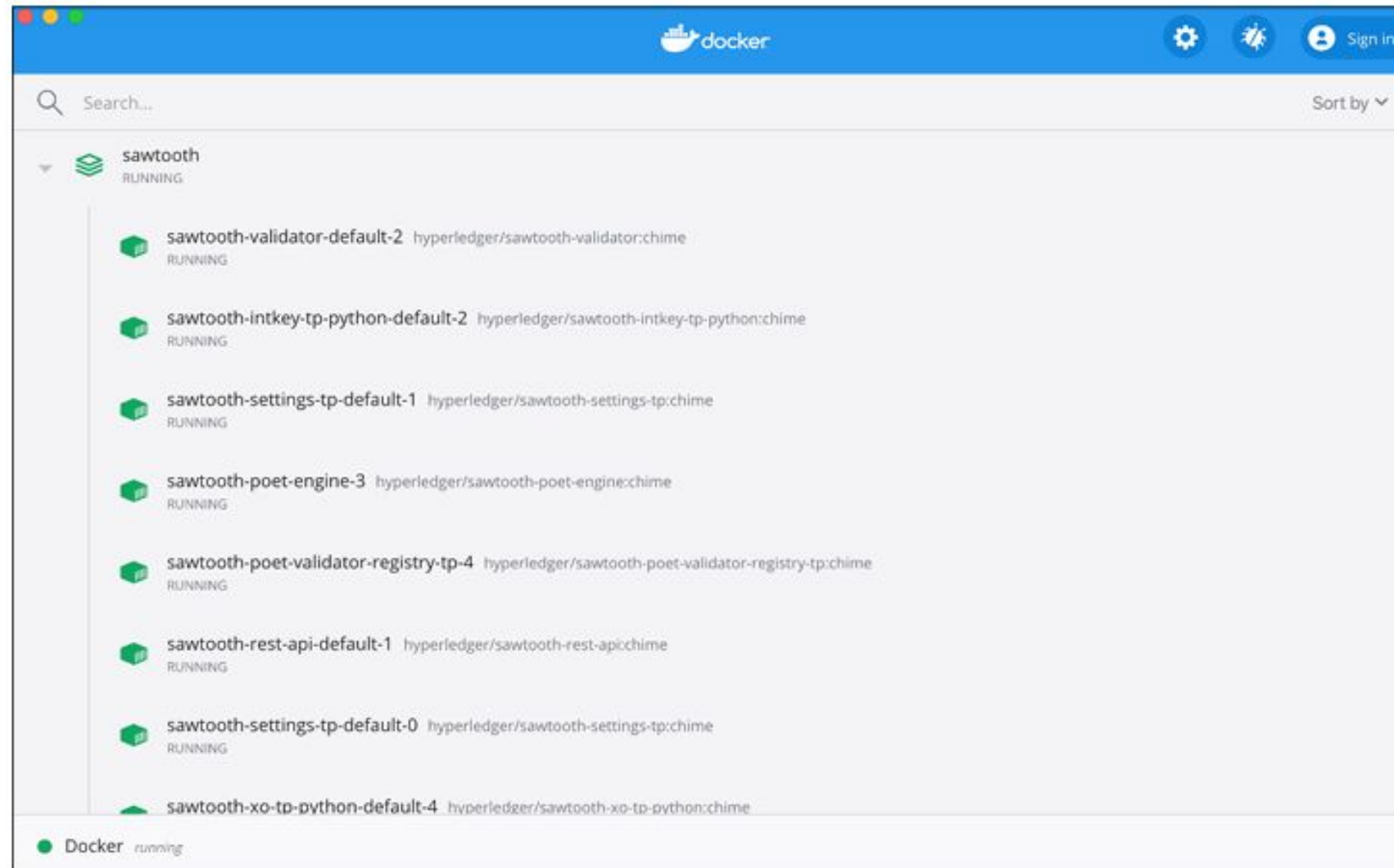


Figure 17.13: Docker dashboard showing the Sawtooth application

6. Confirm that the REST API is running by issuing the following command to connect to the REST API container on a node:

```
$ docker exec -it sawtooth-rest-api-default-0 bash
```

This will open a prompt like the following:

```
root@ea64259cf407:/#
```

7. Issue the command as shown here:

```
root@ea64259cf407:/# ps -eaf | grep 8008
```

This will show an output similar to the following, showing that a process is running for the REST API:

```
root          1      0  0 15:13 ?           00:00:01  
/usr/bin/python3 /usr/bin/sawtooth-rest-api --connect tcp://  
validator-0:4004 --bind rest-api-0:8008
```

Network functionality can be verified by using the following commands:

8. Connect to the shell container:

```
$ docker exec -it sawtooth-shell-default bash
```

This will open a prompt like the one shown here:

```
root@3893ec9be013:/#
```

9. Check whether the peers are up:

```
# curl http://sawtooth-rest-api-default-0:8008/peers
```

This will produce the following output, which displays the list of peers on the network:

```
{
  "data": [
    "tcp://validator-1:8800",
    "tcp://validator-3:8800",
    "tcp://validator-2:8800",
    "tcp://validator-4:8800"
  ],
  "link": http://sawtooth-rest-api-default-0:8008/peers
}
```

10. Another way to find peers is to use the sawtooth command:

```
# sawtooth peer list --url http://sawtooth-rest-api-default-0:8008
```

This command shows an output similar to the following. It shows the list of peers indicating the protocol used, the name of the validator, and the port:

```
tcp://validator-1:8800,tcp://validator-2:8800,tcp://validator-3:8800,tcp://validator-4:8800
```

11. The peer list can also be retried using this command:

```
# sawnet peers list http://sawtooth-rest-api-default-0:8008
```


This command will show the output shown here, which shows that there are other validators available and running:

```
{
  "tcp://validator-0:8800": [
    "tcp://validator-1:8800",
    "tcp://validator-2:8800",
    "tcp://validator-3:8800",
    "tcp://validator-4:8800"
  ]
}
```

12. Similarly, the block list can also be retrieved:

```
# sawnet list-blocks http://sawtooth-rest-api-default-0:8008
```

This command will produce the following output, listing the height, ID, and previous hash of the blocks for node 0:

```
-- NODE 0 --
HEIGHT ID PREVIOUS
2 208f39df d22318b9
1 d22318b9 a1116c4f
0 a1116c4f 00000000
```

13. We can also view the Sawtooth settings, as shown in the following command:

```
$ sawtooth settings list --url http://sawtooth-rest-api-default-0:8008
```

This command shows all the settings of the blockchain:

```
sawtooth.consensus.algorithm.name: PoET
sawtooth.consensus.algorithm.version: 0.1
sawtooth.poet.initial_wait_time: 25
sawtooth.poet.report_public_key_pem:
-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAfMvZzi8GT+li9KeZiInn
4CvFTiuyid+IN4dP1+m...
sawtooth.poet.target_wait_time: 5
sawtooth.poet.valid_enclave_basenames:
b785c58b77152cbe7fd55ee3851c4990000000000000000000000000000000000
sawtooth.poet.valid_enclave_measurements:
c99f21955e38dbb03d2ca838d3af6e43ef438926ed02db4cc729380c8c7a174e
sawtooth.publisher.max_batches_per_block: 100
sawtooth.settings.vote.authorized_keys:
027ea24d40aa8d01ade0d9c6909e6bb52e5a4f04f9c7f3ea3c67436eadddfa4b9
```

14. The block list can also be retrieved using the command shown here:

```
# sawtooth block list --url http://sawtooth-rest-api-default-0:8008
```

The output of this command displays the number, block IDs, batches, transactions and the signer:

NUM	BLOCK_ID	BATS	TXNS	SIGNER
2	208f39dfb18f264e416cd70ec262c9854ecef9aff6dd0ff473eae7e8eccfa74a0685c411a5c4d0e410d2e3d40deed54f3fe355e5498c7a0b051ab6277b62faac			
1	1	032bccb685d25dc2b0a892d283b72bee15d997e76949d66a...		
1	d22318b9f9741386e273f32c0ed76ad3360e4c619cb5a16d74b52ec9e729b93b561b539c02af46729ee00eda1c89dec8fe65aaa1cfb55beecc5d1acf8960416a			
4	4	027ea24d40aa8d01ade0d9c6909e6bb52e5a4f04f9c7f3ea...		
0	a1116c4f18b44da5f9ea1c1517fee2029be900b054898c72a2cd99c907493fed436d7446cbc132715b0649fba247c1ae70423470edc36dee0aef508a9602800b			
4	10	027ea24d40aa8d01ade0d9c6909e6bb52e5a4f04f9c7f3ea...		

15. Specific blocks can be queried using the command:

```
$ sawtooth block show
a1116c4f18b44da5f9ea1c1517fee2029be900b054898c72a2cd99c907493
fed436d7446cbc132715b0649fba247c1ae70423470edc36dee0aef508a9602
800b --url http://sawtooth-rest-api-default-0:8008
```

This will show an output similar to the one shown here. The output is truncated for brevity:

```
batches:
- header:
  signer_public_key:
02071c9bdcd1d42440fdc3655f63561907a396fd0f39d569968114a04dcf801375
  transaction_ids:
  - bba77767e4ceaa405c2d6f33534d4aea840874ed8d577e1f51e1a9bb603451
3250f56c4d67cf860cfd4540028d61374274b26d09bc6583cd120447611134584f
  header_signature:
5d9a20eff9cca693cb9e146c6285ddf3fd6a8e769e9628c79e5592464924265c550f
d33d2581e5aa244a90fa6c7f15e06d1c16fbc2cf2f1b5ae13899d6c3902d
  trace: false
  transactions:
  - header:
```

This output shows the block structure with several fields including the header, transactions, state root hash, and many more.